```c
/*
 * grama_jyothi_clean.c
 * 100% Warning-Free & Error-Free Portable ANSI C Solution
 * Compatible with GCC, Clang, Dev-C++, Code::Blocks, MinGW, and Online C
Compilers.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <float.h>

/* --- TASK 3 & 5: TARIFF CONSTANTS & SAFE MACROS --- */
#define SLAB1_UNITS         100
#define SLAB2_UNITS         200
#define RATE1_PAISE         0L      /* First 100 units free */
#define RATE2_PAISE         225L    /* Units 101-200: Rs 2.25/unit */
#define RATE3_PAISE         450L    /* Above 200 units: Rs 4.50/unit */
#define FIXED_CHARGE_PAISE  5000L   /* Rs 50.00 fixed charge */
#define MAX_VALID_UNITS     10000U

/* --- TASK 4: FULLY PARENTHESIZED MACROS --- */
#define UNSAFE_SQUARE(x)    x * x
#define SQUARE(x)           ((x) * (x))
#define MAX(a, b)           ((a) > (b) ? (a) : (b))
#define MIN(a, b)           ((a) < (b) ? (a) : (b))
#define RUPEES_TO_PAISE(r)  ((long)((r) * 100))
#define PAISE_TO_RUPEES(p)  ((p) / 100)
#define PAISE_TO_REM(p)     ((p) % 100)

typedef unsigned short meter_t;

typedef enum {
    CAT_DOMESTIC = 1,
    CAT_COMMERCIAL = 2,
    CAT_AGRICULTURAL = 3
} consumer_category_t;

typedef struct {
    unsigned int household_id;
    consumer_category_t category;
    meter_t prev_reading;
    meter_t curr_reading;
    unsigned int units_consumed;
    long fixed_charge_paise;
    long energy_charge_paise;
    long total_bill_paise;
} bill_record_t;

/* --- TASK 2: 16-BIT ROLLOVER CONSUMPTION WITHOUT IF-ELSE --- */
unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}

/* --- TASK 3: SLAB-BY-SLAB BILL COMPUTATION IN PAISE --- */
long compute_bill(int units)
{
```

```
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }
    if (units > 0) {
        paise += (long)units * RATE1_PAISE;
    }
    return paise;
}

/* --- TASK 4: SAFE FUNCTION ALTERNATIVE TO MACRO --- */
int safe_max_fn(int a, int b)
{
    return a > b ? a : b;
}

/* --- TASK 7: HALF-UP INTEGER ROUNDING --- */
long round_half_up_paise(long numer, long denom)
{
    return (numer + (denom / 2)) / denom;
}

/* --- TASK 9: VALIDATE & BUILD BILL --- */
int validate_and_build_bill(unsigned int hh_id, int cat, long prev_in,
long curr_in, bill_record_t *out_rec)
{
    meter_t prev;
    meter_t curr;
    unsigned int units;

    if (prev_in < 0 || prev_in > (long)USHRT_MAX || curr_in < 0 ||
curr_in > (long)USHRT_MAX)
        return -1;
    if (cat < CAT_DOMESTIC || cat > CAT_AGRICULTURAL)
        return -2;

    prev = (meter_t)prev_in;
    curr = (meter_t)curr_in;
    units = consumption(prev, curr);

    if (units > MAX_VALID_UNITS)
        return -3;

    out_rec->household_id = hh_id;
    out_rec->category = (consumer_category_t)cat;
    out_rec->prev_reading = prev;
    out_rec->curr_reading = curr;
    out_rec->units_consumed = units;
    out_rec->fixed_charge_paise = FIXED_CHARGE_PAISE;
    out_rec->total_bill_paise = compute_bill((int)units);
    out_rec->energy_charge_paise = out_rec->total_bill_paise -
FIXED_CHARGE_PAISE;
    return 0;
```

```
}

void print_bill_row(const bill_record_t *rec)
{
    printf("| HH-%04u | Cat %d | %5u | %5u | %5u | Rs %4ld.%02ld | Rs
%5ld.%02ld | Rs %5ld.%02ld |\n",
           rec->household_id, (int)rec->category,
           (unsigned int)rec->prev_reading, (unsigned int)rec-
>curr_reading,
           rec->units_consumed,
           PAISE_TO_RUPEES(rec->fixed_charge_paise), PAISE_TO_REM(rec-
>fixed_charge_paise),
           PAISE_TO_RUPEES(rec->energy_charge_paise), PAISE_TO_REM(rec-
>energy_charge_paise),
           PAISE_TO_RUPEES(rec->total_bill_paise), PAISE_TO_REM(rec-
>total_bill_paise));
}

/* --- TASK 10: VILLAGE, TALUK & DISTRICT SCALE BENCHMARK --- */
void test_scale(long num_houses, const char *scale_name)
{
    const long sample_paise[4] = { 5000L, 16255L, 50225L, 95240L };
    float float_rupees = 0.0f;
    double exact_paise = 0.0;
    unsigned int u32_paise = 0U;
    int s32_paise = 0;
    long i;

    for (i = 0; i < num_houses; i++) {
        long p = sample_paise[i % 4];
        float_rupees += (float)p / 100.0f;
        exact_paise  += (double)p;
        u32_paise    += (unsigned int)p;
    }
    s32_paise = (int)u32_paise;

    printf("  %-8s (%6ld houses): Exact = Rs %12.2f | 32-bit int = %12d
(%s) | float err = Rs %+.2f\n",
           scale_name, num_houses, exact_paise / 100.0,
           s32_paise, ((double)s32_paise != exact_paise) ? "OVERFLOW!" :
"OK",
           (double)float_rupees - (exact_paise / 100.0));
}

/* --- TASK 12: SMART METER (15-MIN WH) & TIME-OF-DAY (+20% PEAK) --- */
void run_task12_smart_meter(void)
{
    unsigned long slab1_wh = 100000UL, slab2_wh = 200000UL;
    long rate1_p = 0L, rate2_p = 225L, rate3_p = 450L, fixed_p = 5000L;
    int peak_start = 18, peak_end = 22, peak_pct = 20;
    unsigned long total_wh = 0UL, peak_wh = 0UL;
    unsigned long rem;
    double numer = 0.0;
    long base_paise, tod_paise, final_paise;
    int day, idx;

    for (day = 0; day < 30; day++) {
        for (idx = 0; idx < 96; idx++) {
```

```
            int hr = idx / 4;
            unsigned long wh = (hr >= peak_start && hr < peak_end) ?
150UL : 75UL;
            total_wh += wh;
            if (hr >= peak_start && hr < peak_end) {
                peak_wh += wh;
            }
        }
    }

    rem = total_wh;
    if (rem > slab2_wh) { numer += (double)(rem - slab2_wh) *
(double)rate3_p; rem = slab2_wh; }
    if (rem > slab1_wh) { numer += (double)(rem - slab1_wh) *
(double)rate2_p; rem = slab1_wh; }
    if (rem > 0UL)      { numer += (double)rem * (double)rate1_p; }

    base_paise  = (long)((numer + 500.0) / 1000.0);
    tod_paise   = (long)(((double)base_paise * (double)peak_wh *
(double)peak_pct + ((double)total_wh * 50.0)) / ((double)total_wh *
100.0));
    final_paise = fixed_p + base_paise + tod_paise;

    printf("[TASK 12] Smart Meter (30 Days = 2,880 x 15-min Wh Readings,
+20%% Peak 18:00-22:00):\n");
    printf("  Total Wh: %lu Wh (%.1f kWh) | Peak Wh: %lu Wh (%.1f
kWh)\n",
           total_wh, (double)total_wh / 1000.0,
           peak_wh, (double)peak_wh / 1000.0);
    printf("  Fixed: Rs %ld.%02ld | Base Energy: Rs %ld.%02ld | ToD Peak
Surcharge: Rs %ld.%02ld | Final Bill: Rs %ld.%02ld\n",
           fixed_p / 100L, fixed_p % 100L,
           base_paise / 100L, base_paise % 100L,
           tod_paise / 100L, tod_paise % 100L,
           final_paise / 100L, final_paise % 100L);
}

int main(void)
{
    int i, k;
    float f_tot = 0.0f;
    long p_tot = 0L;
    int tests[5] = {0, 100, 150, 200, 350};
    int a = 4, i_v = 5, j_v = 3;
    int m1, m2;
    int d_houses = 800000;
    int avg_p = 95000;
    int overflowed_32;
    const char *inputs[9] = {
        "1001 1 1200 1200", "1002 1 1000 1100", "1003 1 1000 1150",
        "1004 1 1000 1200", "1005 1 1000 1350", "1006 1 65400 120",
        "1007 1 ABC 1500",  "1008 1 -50 200",   "1009 1 1500 1200"
    };
    struct { unsigned int hh; meter_t p; meter_t c; } batch[5] = {
        {2001, 500, 580}, {2002, 1200, 1350}, {2003, 3000, 3200}, {2004,
10000, 10350}, {2005, 65400, 120}
    };
    long batch_tot = 0L;
```

```
    /* TASK 2 */
    for (i = 0; i < 4000; i++) {
        f_tot += 162.55f;
        p_tot += 16255L;
    }
    printf("[TASK 2] 16-Bit Meter Rollover & Float Drift Proof:\n");
    printf("  sizeof(meter_t) = %lu bytes, maximum reading = %u\n",
           (unsigned long)sizeof(meter_t), (unsigned int)USHRT_MAX);
    printf("  Consumption(1200, 1530)  = %u units\n", consumption(1200,
1530));
    printf("  Consumption(65400, 120)  = %u units  /* rolled over across
65535 */\n", consumption(65400, 120));
    printf("  4,000 Bills of Rs 162.55 -> Float: Rs %.2f vs Exact Paise:
Rs %ld.%02ld (Error: Rs %+.2f)\n\n",
          f_tot, p_tot / 100L, p_tot % 100L, (double)f_tot -
(double)p_tot / 100.0);

    /* TASK 3 */
    printf("[TASK 3] Slab-Wise Tariff Verification (RATE2_PAISE =
%ld):\n", RATE2_PAISE);
    for (i = 0; i < 5; i++) {
        long p = compute_bill(tests[i]);
        printf("  %3d units -> Rs %ld.%02ld\n", tests[i], p / 100L, p %
100L);
    }
    printf("\n");

    /* TASK 4 & 7 */
    printf("[TASK 4 & 7] Safe Macros, Functions & Type Conversion
Surprises:\n");
    printf("  UNSAFE_SQUARE(4+1) = %d vs SAFE SQUARE(4+1) = %d\n",
UNSAFE_SQUARE(a + 1), SQUARE(a + 1));
    m1 = MAX(i_v++, j_v);
    printf("  MAX(i++, j) macro (i=5, j=3)  -> result=%d, i became %d
(incremented twice!)\n", m1, i_v);
    i_v = 5; j_v = 3;
    m2 = safe_max_fn(i_v++, j_v);
    printf("  safe_max_fn(i++, j) (i=5,j=3) -> result=%d, i became %d
(incremented once)\n", m2, i_v);
    printf("  Surprise 1 (Int Div)  : (150/100)*2.25 = %.2f vs
((double)150/100)*2.25 = %.4f\n",
          (150 / 100) * 2.25, ((double)150 / 100.0) * 2.25);
    overflowed_32 = (int)((unsigned int)d_houses * (unsigned int)avg_p);
    printf("  Surprise 3 (Overflow) : 800000*95000 (32-bit int) = %d vs
exact = %.0f\n\n",
          overflowed_32, (double)d_houses * (double)avg_p);

    /* TASK 6 & 9 */
    printf("[TASK 6 & 9] Comprehensive Validation Suite & 5-Bill Hand-
Calculated Batch:\n");
    printf("| HH ID   | Cat   |  Prev |  Curr | Units | Fixed Chg  |
Energy Chg  | Total Bill  |\n");
    printf("|---------|-------|-------|-------|-------|------------|-----
--------|-------------|\n");
    for (k = 0; k < 9; k++) {
        unsigned int hh;
        int cat;
```

```
        long pr, cu;
        bill_record_t r;
        int st;
        if (sscanf(inputs[k], "%u %d %ld %ld", &hh, &cat, &pr, &cu) != 4)
{
            printf("| [REJECTED INPUT] \"%s\" -> Format error (non-
numeric characters)\n", inputs[k]);
            continue;
        }
        st = validate_and_build_bill(hh, cat, pr, cu, &r);
        if (st == 0) print_bill_row(&r);
        else if (st == -1) printf("| [REJECTED INPUT] \"%s\" -> Out-of-
range reading (must be 0..65535)\n", inputs[k]);
        else if (st == -3) printf("| [REJECTED INPUT] \"%s\" -> Current <
Previous without rollover\n", inputs[k]);
    }

    printf("|---------|-------|-------|-------|-------|------------|-----
--------|-------------|\n");
    for (i = 0; i < 5; i++) {
        bill_record_t r;
        validate_and_build_bill(batch[i].hh, CAT_DOMESTIC, batch[i].p,
batch[i].c, &r);
        print_bill_row(&r);
        batch_tot += r.total_bill_paise;
    }
    printf("  >> 5-Bill Batch Grand Total: %ld paise = Rs %ld.%02ld
(Hand-Calculated: Rs 1964.50)\n\n",
           batch_tot, batch_tot / 100L, batch_tot % 100L);

    /* TASK 10 */
    printf("[TASK 10] Village (4,000), Taluk (60,000) & District
(800,000) Scale Benchmark:\n");
    test_scale(4000L,   "Village");
    test_scale(60000L,  "Taluk");
    test_scale(800000L, "District");
    printf("\n");

    /* TASK 12 */
    run_task12_smart_meter();
    return 0;
}
```